

Taller de SQL #1

Del Álgebra Relacional al SQL

Bases de Datos (Relacionales)

Ing. Informática · Lic. en Cs. de Datos

Ricardo Di Pasquale
Alejandro Isidro
Edgardo Sanchez
Delfina Las Heras

AGENDA

1. SQL como derivado del Álgebra Relacional
2. Esquema del taller
3. Unarios: proyección, selección, renombrado
4. Producto cartesiano y juntas
5. Conjuntos: unión, intersección, diferencia
6. Cociente y funciones agregadas
7. SELECT completo y vistas

LA IDEA DEL TALLER

SQL, un derivado del Álgebra Relacional

Cada operador tiene su cláusula SQL

Operador	Símbolo	En SQL
Proyección	π	SELECT [DISTINCT] columnas
Selección	σ	WHERE
Renombrado	ρ / $\leftarrow$	AS (alias de tabla o columna)
Producto cartesiano	$\times$	FROM a, b / CROSS JOIN
Junta natural	$\bowtie$	NATURAL JOIN
Junta interna (theta)	$\bowtie$ (cond.)	INNER JOIN ... ON
Junta externa izq / der	$\bowtie$ izq/der	LEFT JOIN / RIGHT JOIN
Unión · intersección · diferencia	$\cup$ $\cap$ $-$	UNION · INTERSECT · MINUS/EXCEPT
Cociente	$/$	double NOT EXISTS
Agregación	$\mathcal{F}$	GROUP BY ... (COUNT, AVG, ...)

Esquema del taller

Tabla (letra)	Columnas
CLIENTES (C)	NroCliente, Nombre, Dirección, Teléfono, Ciudad
PRODUCTO (P)	CodProducto, Descripción, Precio
VENTA (V)	CodProducto, NroCliente, Cantidad, IdVenta

En las fórmulas de AR usamos C, P, V; en SQL, los nombres completos. VENTA relaciona clientes con productos.

Proyección — π

$$\pi_{\text{Nombre, Ciudad}}(C)$$

Idea: elige columnas (recorte vertical); en AR elimina duplicados.

```
SELECT DISTINCT Nombre, Ciudad  
FROM CLIENTES;
```

La proyección del AR es de conjuntos → por eso DISTINCT. Sin él, SQL conserva duplicados.

Selección — σ

$\sigma_{\text{Ciudad} = \text{'Córdoba'}} (C)$

Idea: elige filas (recorte horizontal) que cumplen un predicado.

```
SELECT *  
FROM CLIENTES  
WHERE Ciudad = 'Córdoba';
```

$\sigma \leftrightarrow$ WHERE. El predicado combina comparaciones con AND / OR / NOT.

Renombrado — p (o $\leftarrow$)

p_{Cli} (CLIENTES) PrecioUnit $\leftarrow$ Precio

Idea: da un nombre alternativo a una relación o a un atributo (clave para las auto-juntas).

```
SELECT c.Nombre, p.Precio AS PrecioUnit
FROM CLIENTES AS c, PRODUCTO AS p;
-- alias de tabla (AS c) y de columna (AS PrecioUnit)
```

En SQL, p se expresa con AS (el AS es opcional: FROM CLIENTES c también vale).

Producto cartesiano — $\times$

$C \times P$

Idea: combina cada fila de C con cada fila de P ($|C| \cdot |P|$ filas).

```
SELECT * FROM CLIENTES, PRODUCTO;      -- coma sin WHERE = producto cartesiano
SELECT * FROM CLIENTES CROSS JOIN PRODUCTO;  -- forma explícita equivalente
```

La lista de tablas separadas por coma, sin condición, ES un producto cartesiano válido.

Junta interna natural — ⚡

V ⚡ C

Idea: junta por TODOS los atributos de igual nombre y los fusiona (aquí, NroCliente).

```
SELECT *  
FROM VENTA  
NATURAL JOIN CLIENTES;
```

Usa todos los atributos comunes; aquí el único común es NroCliente.

Junta interna (theta) — $\bowtie$

V $\bowtie$ P
V.CodProducto = P.CodProducto

Idea: junta interna con la condición como subíndice del símbolo.

```
SELECT *  
FROM VENTA v  
INNER JOIN PRODUCTO p ON v.CodProducto = p.CodProducto;
```

La condición puede ser =, >, <, ≠... Con = es un equijoin y conserva ambas columnas.

Junta externa — izquierda / derecha

C ^{izq}  V
NroCliente

C ^{der}  V
NroCliente

Idea: conservan las filas sin pareja del lado indicado, rellenando con NULL.

```
-- todos los clientes, tengan o no ventas:  
SELECT * FROM CLIENTES c LEFT JOIN VENTA v ON c.NroCliente = v.NroCliente;  
-- todas las ventas, exista o no el cliente:  
SELECT * FROM CLIENTES c RIGHT JOIN VENTA v ON c.NroCliente = v.NroCliente;
```

Unión (conjuntos) — U

$$\sigma_{\text{Ciudad}='Córdoba'}(C) \cup \sigma_{\text{Ciudad}='Rosario'}(C)$$

Idea: une dos relaciones compatibles; en AR es de conjuntos (sin duplicados).

```
SELECT * FROM CLIENTES WHERE Ciudad = 'Córdoba'  
UNION  
SELECT * FROM CLIENTES WHERE Ciudad = 'Rosario';
```

UNION elimina duplicados (como el AR). UNION ALL los conserva (multiconjunto).

Intersección — $\cap$

$$\pi_{\text{NroCliente}} \sigma_{\text{CodProducto}=10}(V) \cap \pi_{\text{NroCliente}} \sigma_{\text{CodProducto}=20}(V)$$

Idea: clientes que compraron el producto 10 Y el 20.

```
-- con INTERSECT:
SELECT NroCliente FROM VENTA WHERE CodProducto = 10
INTERSECT
SELECT NroCliente FROM VENTA WHERE CodProducto = 20;
-- alternativa con IN (motores sin INTERSECT):
SELECT DISTINCT NroCliente FROM VENTA WHERE CodProducto = 10
AND NroCliente IN (SELECT NroCliente FROM VENTA WHERE CodProducto = 20);
```

Diferencia — – (MINUS / EXCEPT)

$$\pi_{\text{NroCliente}}(C) - \pi_{\text{NroCliente}}(V)$$

Idea: clientes que NUNCA compraron (están en C y no en V).

```
SELECT NroCliente FROM CLIENTES
MINUS
SELECT NroCliente FROM VENTA;
-- Oracle
-- SQL estándar: usar EXCEPT en lugar de MINUS
```

Oracle usa MINUS; el SQL estándar usa EXCEPT.

Diferencia — alternativas sin operador de conjunto

$$\pi_{\text{NroCliente}}(C) - \pi_{\text{NroCliente}}(V) \text{ (mismo resultado)}$$

Idea: para motores sin MINUS / EXCEPT.

```
-- NOT IN:  
SELECT NroCliente FROM CLIENTES  
WHERE NroCliente NOT IN (SELECT NroCliente FROM VENTA);  
  
-- NOT EXISTS:  
SELECT NroCliente FROM CLIENTES c  
WHERE NOT EXISTS (SELECT 1 FROM VENTA v  
                  WHERE v.NroCliente = c.NroCliente);
```

Ojo: NOT IN falla si la subconsulta trae NULL; NOT EXISTS es más seguro.

Cociente (/) — A / B: “X con TODOS los Y de B”

$$A / B = \pi_x(A) - \pi_x((\pi_x(A) \times B) - A)$$

Idea: el cociente devuelve los X asociados con TODOS los Y de la tabla divisora B.

A: X1→{Y1, Y2, Y3} X2→{Y2, Y3} X3→{Y1, Y3}

B: {Y1, Y2, Y3}

A / B = { X1 } (X1 tiene todos los Y de B; X2 y X3 no).

SQL no tiene operador de división: se resuelve con doble NOT EXISTS (slide siguiente).

Cociente (/) — en SQL (doble NOT EXISTS)

$$A / B = \pi_x(A) - \pi_x((\pi_x(A) \times B) - A)$$

Idea: “no hay ningún Y de B que le falte a ese X”.

```
SELECT DISTINCT X FROM A AS A1
WHERE NOT EXISTS (
  SELECT 1 FROM A AS A2, B
  WHERE A1.X = A2.X AND NOT EXISTS (
    SELECT 1 FROM A AS A3
    WHERE A3.X = A2.X AND A3.Y = B.Y));
```

Funciones agregadas — $\mathcal{F}$

CodProducto $\mathcal{F}$ COUNT(*), SUM(Cantidad) (V)

Idea: agrupa y calcula funciones; SIN GROUP BY resume toda la tabla.

```
-- sin agrupar: una sola fila con el resumen de TODAS las filas
SELECT COUNT(*) AS total, AVG(Cantidad) AS prom FROM VENTA;
-- agrupando por producto:
SELECT CodProducto, COUNT(*), SUM(Cantidad)
FROM VENTA
GROUP BY CodProducto
HAVING SUM(Cantidad) > 100;
```

Izq. de $\mathcal{F}$ = GROUP BY; der. = las funciones. HAVING filtra grupos (σ sobre el resultado).

Sintaxis mínima de un SELECT completo

```
SELECT      columnas / funciones
FROM        tabla
INNER JOIN  otra  ON  condición
WHERE       filtro de filas
GROUP BY    columnas
HAVING      filtro de grupos
ORDER BY    columnas [ASC | DESC];
```

Orden lógico de evaluación: FROM/JOIN → WHERE → GROUP BY → HAVING → SELECT → ORDER BY.

Vistas: relaciones con nombre

Una vista es una consulta guardada con nombre que se usa como si fuera una tabla.

```
CREATE VIEW ventas_detalle AS
SELECT v.IdVenta, c.Nombre, p.Descripción, v.Cantidad, p.Precio
FROM VENTA v
INNER JOIN CLIENTES c ON c.NroCliente = v.NroCliente
INNER JOIN PRODUCTO p ON p.CodProducto = v.CodProducto;
```

- ▶ Nombran subproblemas: igual que en AR usamos relaciones intermedias para modelar por partes.
- ▶ Simplifican consultas complejas y dan una capa lógica / de seguridad.
- ▶ No abusar: encadenar vistas sobre vistas complica el rendimiento y el mantenimiento.

Para recordar

- ▶ $\pi \rightarrow$ SELECT (DISTINCT); $\sigma \rightarrow$ WHERE; $\rho \rightarrow$ alias AS.
- ▶ Junta = producto + selección; la natural fusiona los atributos comunes.
- ▶ Interna: INNER JOIN ... ON (la condición va como subíndice en AR).
- ▶ Externas: LEFT JOIN / RIGHT JOIN conservan el lado indicado (NULL en el otro).
- ▶ Conjuntos: UNION · INTERSECT (o IN) · MINUS/EXCEPT (o NOT IN / NOT EXISTS).
- ▶ Cociente A/B = doble NOT EXISTS (“X con todos los Y de B”).
- ▶ $\mathcal{F} \rightarrow$ GROUP BY + funciones; sin grupo, resume toda la tabla; HAVING filtra grupos.
- ▶ Las vistas nombran subproblemas: útiles, pero no conviene abusar.

¡Muchas gracias! Q&A

Seguinos



@ingeagrariasuca



@IngeagrariasUCA



Ingeniería y Ciencias Agrarias

Contacto



Av. Alicia Moreau de Justo 1500
C1107AAZ, Buenos Aires, Argentina



0810-220-0822



inge_agrarias@uca.edu.ar